

# 第 15 章

## 模板匹配

模板匹配是指在当前图像 A 内寻找与图像 B 最相似的部分,一般将图像 A 称为输入图像,将图像 B 称为模板图像。模板匹配的操作方法是将模板图像 B 在图像 A 上滑动,遍历所有像素以完成匹配。

例如,在图 15-1 中,希望在图中的大图像“lena”内寻找左上角的“眼睛”图像。此时,大图像“lena”是输入图像,“眼睛”图像是模板图像。查找的方式是,将模板图像在输入图像内从左上角开始滑动,逐个像素遍历整幅输入图像,以查找与其最匹配的部分。



图 15-1 模板匹配示例

### 15.1 模板匹配基础

在 OpenCV 内,模板匹配是使用函数 `cv2.matchTemplate()` 实现的。该函数的语法格式为:

```
result = cv2.matchTemplate(image, templ, method[, mask ] )
```

其中:

- `image` 为原始图像,必须是 8 位或者 32 位的浮点型图像。
- `templ` 为模板图像。它的尺寸必须小于或等于原始图像,并且与原始图像具有同样的类型。
- `method` 为匹配方法。该参数通过 `TemplateMatchModes` 实现,有 6 种可能的值,如表 15-1 所示。



表 15-1 method 的值以及含义

| 参数值                  | 对应数值 | 说明                                                                               |
|----------------------|------|----------------------------------------------------------------------------------|
| cv2.TM_SQDIFF        | 0    | 以方差为依据进行匹配。若完全匹配，则结果为零；若不匹配，则会得到一个很大的值                                           |
| cv2.TM_SQDIFF_NORMED | 1    | 标准（归一化）平方差匹配                                                                     |
| cv2.TM_CCORR         | 2    | 相关匹配，这类方法将模板图像与输入图像相乘，如果乘积较大，表示匹配程度较高；如果乘积为 0，则表示匹配效果最差                          |
| cv2.TM_CCORR_NORMED  | 3    | 标准（归一化）相关匹配                                                                      |
| cv2.TM_CCOEFF        | 4    | 相关系数匹配，这类方法将模板图像与其均值的相对值，和输入图像与其均值的相关值进行匹配。1 表示完美匹配，-1 表示糟糕的匹配，0 表示没有任何相关性（随机序列） |
| cv2.TM_CCOEFF_NORMED | 5    | 标准（归一化）相关系数匹配                                                                    |

其具体对应的计算公式如表 15-2 所示。

表 15-2 method值具体对应的公式关系<sup>1</sup>

| 值                    | 公式                                                                                                                                                                                                                         |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cv2.TM_SQDIFF        | $R(x,y) = \sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2$                                                                                                                                                                        |
| cv2.TM_SQDIFF_NORMED | $R(x,y) = \frac{\sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$                                                                                               |
| cv2.TM_CCORR         | $R(x,y) = \sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))$                                                                                                                                                                      |
| cv2.TM_CCORR_NORMED  | $R(x,y) = \frac{\sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$                                                                                             |
| cv2.TM_CCOEFF        | $R(x,y) = \sum_{x',y'} (T'(x',y') \cdot I'(x+x',y+y'))$ <p>当</p> $T'(x',y') = T(x',y') - 1/(w \cdot h) \cdot \sum_{x'',y''} T(x'',y'')$ $I'(x+x',y+y') = I(x+x',y+y') - 1/(w \cdot h) \cdot \sum_{x'',y''} I(x+x'',y+y'')$ |
| cv2.TM_CCOEFF_NORMED | $R(x,y) = \frac{\sum_{x',y'} (T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$                                                                                         |

- mask 为模板图像掩模。它必须和模板图像 templ 具有相同的类型和大小。通常情况下该值使用默认值即可。当前，该参数仅支持 TM\_SQDIFF 和 TM\_CCORR\_NORMED 两个值。

函数 cv2.matchTemplate()的返回值 result 是由每个位置的比较结果组合所构成的一个结果集，类型是单通道 32 位浮点型。如果输入图像（原始图像）尺寸是 W\*H，模板的尺寸是 w\*h，则返回值的大小为(W-w+1)\*(H-h+1)。

在进行模板匹配时，模板在原始图像内遍历。在水平方向上：

1 I 表示输入图像，T 表示模板，R 表示输出的结果图像，x 和 y 表示位置信息。

- 遍历的起始坐标是原始图像左数第 1 个像素值（序号从 1 开始）。
- 最后一次比较是当模板图像位于原始图像的最右侧时，此时其左上角像素点所在的位置是  $W-w+1$ 。

因此，返回值 `result` 在水平方向上的大小是  $W-w+1$ （水平方向上的比较次数）。

在垂直方向上：

- 遍历的起始坐标从原始图像顶端的第 1 个像素开始。
- 最后一次比较是当模板图像位于原始图像的最下端时，此时其左上角像素点所在位置是  $H-h+1$ 。

所以，返回值 `result` 在垂直方向上的大小是  $H-h+1$ （垂直方向上的比较次数）。

如果原始图像尺寸是  $W \times H$ ，模板的尺寸是  $w \times h$ ，则返回值的大小为  $(W-w+1) \times (H-h+1)$ 。也就是说，模板图像要在输入图像内比较  $(W-w+1) \times (H-h+1)$  次。

例如，在图 15-2 中，左上方的  $2 \times 2$  小方块是模板图像，右下方的  $10 \times 10$  图像是输入图像（原始图像）。在进行模板匹配时：

- 首先将模板图像置于输入图像的左上角。
- 模板图像在向右移动时，最远只能位于输入图像的最右侧边界处，此时模板图像左上角的像素对应着输入图像的第 9 列（输入图像宽度-模板图像宽度+1 =  $10-2+1=9$ ）。
- 模板图像在向下移动时，最远只能位于输入图像最下端的边界处。此时模板图像左上角的像素对应着输入图像的第 9 行（输入图像高度-模板图像高度+1 =  $10-2+1=9$ ）。

根据上述分析可知，比较结果 `result` 的大小满足  $(W-w+1) \times (H-h+1)$ ，在上例中就是  $(10-2+1) \times (10-2+1)$ ，即  $9 \times 9$ 。也就是说，模板图像要在输入图像内总计比较  $9 \times 9 = 81$  次，这些比较结果将构成一个  $9 \times 9$  大小的二维数组。

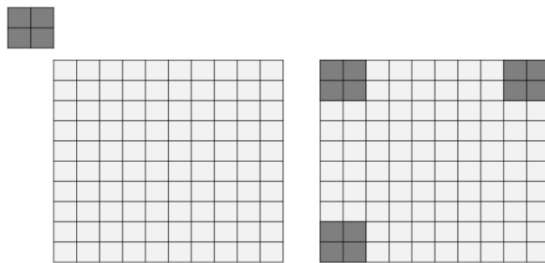


图 15-2 模板图像遍历输入图像的示意图

这里需要注意的是，函数 `cv2.matchTemplate()` 通过参数 `method` 来决定使用不同的查找方法。对于不同的查找方法，返回值 `result` 具有不同的含义。例如：

- `method` 的值为 `cv2.TM_SQDIFF` 和 `cv2.TM_SQDIFF_NORMED` 时，`result` 值为 0 表示匹配度最好，值越大，表示匹配度越差。
- `method` 的值为 `cv2.TM_CCORR`、`cv2.TM_CCORR_NORMED`、`cv2.TM_CCOEFF` 和 `cv2.TM_CCOEFF_NORMED` 时，`result` 的值越小表示匹配度越差，值越大表示匹配度越好。

从上述分析可以看出，查找方法不同，结果的判定方式也不同。在查找最佳匹配时，首先

要确定使用的是何种 `method`，然后再确定到底是查找最大值，还是查找最小值。

查找最值（极值）与最值所在的位置，可以使用 `cv2.minMaxLoc()` 函数实现。该函数语法格式如下：

```
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc( src [, mask] )
```

其中：

- `src` 为单通道数组。
- `minVal` 为返回的最小值，如果没有最小值，则可以是 `NULL`（空值）。
- `maxVal` 为返回的最大值，如果没有最小值，则可以是 `NULL`。
- `minLoc` 为最大值的位置，如果没有最大值，则可以是 `NULL`。
- `maxLoc` 为最大值的位置，如果没有最大值，则可以是 `NULL`。
- `mask` 为用来选取掩模的子集，可选项。

函数 `cv2.minMaxLoc()` 能够查找整个数组内的最值及它们的位置，并且可以根据当前的掩模集来选取特定子集的极值。有关该函数的更多说明及实例，请参考第 12 章。

综上所述，函数 `cv2.matchTemplate()` 返回值中的最值位置就是模板匹配的位置。当然，选用表 15-1 中的不同参数值，匹配位置可能位于最大值所在的位置也可能位于最小值所在的位置。通过函数 `cv2.minMaxLoc()` 来查找函数 `cv2.matchTemplate()` 返回值中的最值位置，就可以找到最佳模板匹配的位置

例如，当 `method` 的值为 `cv2.TM_SQDIFF` 和 `cv2.TM_SQDIFF_NORMED` 时，0 表示最佳匹配，值越大，则表示匹配效果越差。因此，在使用这两种方法时，要寻找最小值所在的位置作为最佳匹配。如下语句能够找到 `cv2.matchTemplate()` 函数返回值中最小值的位置：

```
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(matchTemplate 函数的返回值)
topLeft = minLoc # 查找最小值所在的位置
```

以 `topLeft` 点为模板匹配位置的左上角坐标，结合模板图像的宽度 `w` 和高度 `h` 可以确定匹配位置的右下角坐标，代码如下所示：

```
bottomRight = (topLeft[0] + w, topLeft[1] + h) #w 和 h 是模板图像的宽度和高度
```

当 `method` 的值为 `cv2.TM_CCORR`、`cv2.TM_CCORR_NORMED`、`cv2.TM_CCOEFF` 和 `cv2.TM_CCOEFF_NORMED` 时，`cv2.matchTemplate()` 函数的返回值越小，表示匹配度越差，而返回值越大则表示匹配度越好。此时，要寻找最大值所在的位置作为最佳匹配。如下语句能够找到模板匹配返回值中最大值的位置，并以该点为左上角，结合模板的宽度 `w` 和高度 `h` 确定匹配位置的右下角坐标。

```
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(matchTemplate 函数的返回值)
topLeft = maxLoc # 查找最大值所在的位置
bottomRight = (topLeft[0] + w, topLeft[1] + h) # w 和 h 是模板的宽度和高度
```

通过上述方式，我们确定了模板匹配的矩形对角坐标位置，接下来可以借助函数 `cv2.rectangle()` 将该位置用白色标记出来。函数 `cv2.rectangle` 的语法格式为：

```
Img = cv.rectangle( img, pt1, pt2, color[, thickness])
```

式中各个参数的含义为：

- `img` 表示要标记的目标图像。
- `pt1` 是矩形的顶点。
- `pt2` 是 `pt1` 的对角顶点。
- `color` 是要绘制矩形的颜色或灰度级（灰度图像）。
- `thickness` 是矩形边线的宽度。

因此，使用的标记语句为：

```
cv2.rectangle(img,topLeft, bottomRight, 255, 2)
```

该语句表示，在 `img` 内标记一个矩形，矩形的两个对角顶点为 `topLeft` 和 `bottomRight`，矩形颜色为白色（255），宽度为 2。

**【例 15.1】**使用函数 `cv2.matchTemplate()` 进行模板匹配。要求参数 `method` 的值设置为 `cv2.TM_SQDIFF`，显示函数的返回结果及匹配结果。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
img = cv2.imread('lena512g.bmp',0)
template = cv2.imread('temp.bmp',0)
th, tw = template.shape[::]
rv = cv2.matchTemplate(img,template,cv2.TM_SQDIFF)
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(rv)
topLeft = minLoc
bottomRight = (topLeft[0] + tw, topLeft[1] + th)
cv2.rectangle(img,topLeft, bottomRight, 255, 2)
plt.subplot(121),plt.imshow(rv,cmap = 'gray')
plt.title('Matching Result'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(img,cmap = 'gray')
plt.title('Detected Point'), plt.xticks([]), plt.yticks([])
plt.show()
```

本例中所使用的输入图像是 512×512 的图像 `lena512g.bmp`，模板图像是从输入图像中截取的其眼部子图，如图 15-3 所示。

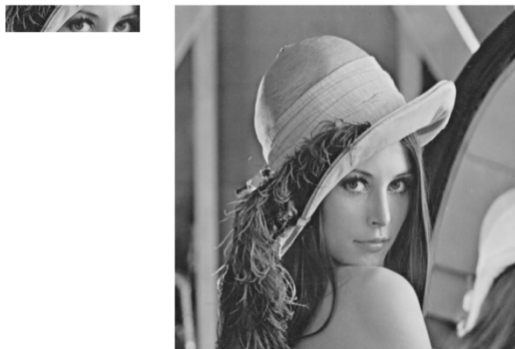


图 15-3 模板图像（左图）和输入图像（右图）

运行上述代码，得到如图 15-4 所示结果，其中左图是函数 `cv2.matchTemplate()` 的返回值，右图是模板匹配的结果。

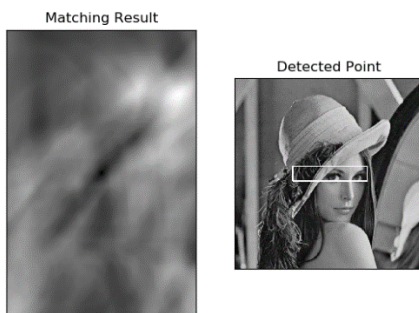


图 15-4 【例 15.1】程序的运行结果

这里需要注意，在计算模板图像的宽度时，使用的语句为：

```
th, tw = template.shape[::]
```

返回值中的 `th` 是模板图像的高度，`tw` 是模板图像的宽度。在 OpenCV 官网的示例中，使用的语句形式为：

```
tw, th = template.shape[::-1]
```

该语句返回的也是模板图像的宽度和高度，只不过语句 `template.shape[::-1]` 将宽度和高度的顺序进行了调换。

下面通过一段代码来说明语句 `template.shape[::-1]` 的含义。例如：

```
import numpy as np
list2d = np.arange(18).reshape(3,6)
print(list2d)
```

能够输出当前的 `list2d` 为：

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]]
```

从其定义及返回值可以观察到，`list2d` 是一个高度为 3 个像素，宽度为 6 个像素的模板图像。首先，使用 `list2d.shape[:]` 返回其 `shape` 值，并将该值打印出来：

```
h,w=list2d.shape[:]
print(h,w)
```

此时，会得到打印结果：“3 6”。返回值中，第 1 个值是高度，第 2 个值是宽度。

接下来，使用 `list2d.shape[::-1]` 返回模板图像的 `shape` 值，并将该值打印出来：

```
w,h=list2d.shape[::-1]
print(w,h)
```

此时，会得到打印结果：“6 3”。返回值中，第 1 个值是宽度，第 2 个值是高度。

**【例 15.2】**使用 `cv2.matchTemplate()` 函数进行模板匹配。要求参数 `method` 的值设置为 `cv2.TM_CCOEFF`，显示函数的返回结果及匹配结果。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
img = cv2.imread('lena512g.bmp',0)
template = cv2.imread('temp.bmp',0)
tw, th = template.shape[::-1]
rv = cv2.matchTemplate(img,template,cv2.TM_CCOEFF)
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(rv)
topLeft = maxLoc
bottomRight = (topLeft[0] + tw, topLeft[1] + th)
cv2.rectangle(img,topLeft, bottomRight, 255, 2)
plt.subplot(121),plt.imshow(rv,cmap = 'gray')
plt.title('Matching Result'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(img,cmap = 'gray')
plt.title('Detected Point'), plt.xticks([]), plt.yticks([])
plt.show()
```

本例使用的输入图像及模板图像与例 15.1 的相同，如图 15-3 所示。

运行上述代码，得到如图 15-5 所示的结果，其中左图是函数 cv2.matchTemplate() 的返回值，右图是模板匹配的结果。

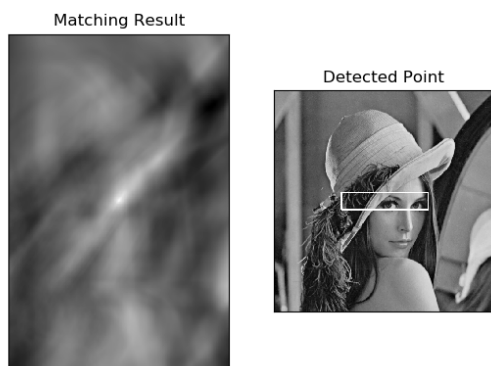


图 15-5 【例 15.2】程序的运行结果

在本节的两个例子中，使用 cv2.matchTemplate() 函数进行模板匹配时，查找最值（极值）的方式是不一样的：

- 在例 15.1 中，参数 method 的值为 cv2.TM\_SQDIFF，查找的是最小值所在的位置。
- 在例 15.2 中，参数 method 的值为 cv2.TM\_CCOEFF，查找的是最大值所在的位置。

## 15.2 多模板匹配

在前面的例子中，我们在输入图像 lena 中搜索其眼部子图，该子图在整个输入图像内仅出现了一次。但是，有些情况下，要搜索的模板图像很可能在输入图像内出现了多次，这时就需要找出多个匹配结果。而函数 cv2.minMaxLoc() 仅仅能够找出最值，无法给出所有匹配区域的



位置信息。所以，要想匹配多个结果，使用函数 `cv2.minMaxLoc()` 是无法实现的，需要利用阈值进行处理。

下面分步骤介绍如何获取多模板匹配的结果。

### 1. 获取匹配位置的集合

函数 `where()` 能够获取模板匹配位置的集合。对于不同的输入，其返回的值是不同的。

- 当输入（参数）是一维数组时，返回值是一维索引，只有一组索引数组。
- 当输入是二维数组时，返回的是匹配值的位置索引，因此会有两组索引数组表示返回的位置。

以下代码查找在一维数组 `a` 中，数值大于 5 的元素的索引（即该元素所在的位置，数组的索引从 0 开始）：

```
import numpy as np
a=np.array([3,6,8,1,2,88])
b=np.where(a>5)
print(b)
```

该段代码返回的结果为：

```
(array([1, 2, 5], dtype=int64),)
```

说明索引值为 1、2、5 的数组元素，它们的值是大于 5 的。

上面介绍的是输入值为一维数组时的情况。当输入值是二维数组时，函数 `where()` 会返回满足条件的值在二维数组中的索引。例如，以下代码查找在二维数组 `am` 中，值大于 5 的元素的索引：

```
import numpy as np
am=np.array([[3,6,8,77,66],[1,2,88,3,98],[11,2,67,5,2]])
b=np.where(am>5)
print(b)
```

该段代码返回的结果为：

```
(array([0, 0, 0, 0, 1, 1, 2, 2], dtype=int64),
 array([1, 2, 3, 4, 2, 4, 0, 2], dtype=int64))
```

上述结果说明，存在二维数组 `am`，它的值为：

```
[[ 3  6  8 77 66]
 [ 1  2 88  3 98]
 [11  2 67  5  2]]
```

其中，位置 [0, 1]、[0, 2]、[0, 3]、[0, 4]、[1, 2]、[1, 4]、[2, 0]、[2, 2] 上的元素值大于 5。

综上所述，函数 `np.where()` 可以找出在函数 `cv2.matchTemplate()` 的返回值中，哪些位置上的值是大于阈值 `threshold` 的。在具体实现时，可以采用的语句为：

```
loc = np.where( res >= threshold)
```

式中：

- `res` 是函数 `cv2.matchTemplate()` 进行模板匹配后的返回值。



- `threshold` 是预设的阈值
- `loc` 是满足 “`res >= threshold`” 的像素点的索引集合。例如，在上面的二维数组 `am` 中，返回的大于 5 的元素索引集合为(`array([0, 0, 0, 0, 1, 1, 2, 2], dtype=int64)`, `array([1, 2, 3, 4, 2, 4, 0, 2], dtype=int64)`)。返回值 `loc` 中的两个元素，分别表示匹配值的行索引和列索引。

## 2. 循环

要处理多个值，通常需要用循环。例如，有一个列表，其中的值为 71、23、16，希望将这些值逐个输出，可以这样写代码：

```
value = [71, 23, 16]
for i in value:
    print('value 内的值:', i)
```

运行上述代码，得到的输出结果为：

```
value 内的值: 71
value 内的值: 23
value 内的值: 16
```

因此，在获取匹配值的索引集合后，可以采用如下语句遍历所有匹配的位置，对这些位置做标记：

```
for i in 匹配位置集合:
    标记匹配位置。
```

## 3. 在循环中使用函数`zip()`

函数 `zip()` 用可迭代的对象作为参数，将对象中对应的元素打包成一个个元组，然后返回由这些元组组成的列表。

例如，以下代码使用函数 `zip()` 将 `t` 内对应的元素打包成一个个元组，并打印了由这些元组组成的列表：

```
x = [1, 2, 3]
y = [4, 5, 6]
z = [7, 8, 9]
t = (x, y, z)
print(t)
for i in zip(*t):
    print(i)
```

上述代码中，语句 `print(t)` 将 `t` 内的元素输出，结果为：

```
([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

循环语句 `for i in zip(*t)` 将 `t` 内的元素打包成元组后输出，结果为：

```
(1, 4, 7)
(2, 5, 8)
(3, 6, 9)
```

因此，如果希望循环遍历由 `np.where()` 返回的模板匹配索引集合，可以采用的语句为：

```
for i in zip(*模板匹配索引集合):
```



标记处理

例如，对于前面提到的数组 `am`，使用函数 `zip()` 循环，就可以得到其中大于 5 的元素索引的集合：

```
import numpy as np
am=np.array([[3,6,8,77,66],[1,2,88,3,98],[11,2,67,5,2]])
print(am)
b=np.where(am>5)
for i in zip(*b):
    print(i)
```

上述代码的输出结果为：

```
[[ 3  6  8 77 66]
 [ 1  2 88  3 98]
 [11  2 67  5  2]]
(0, 1)
(0, 2)
(0, 3)
(0, 4)
(1, 2)
(1, 4)
(2, 0)
(2, 2)
```

#### 4. 调整坐标

函数 `numpy.where()` 可以获取满足条件的模板匹配位置集合，然后可以使用函数 `cv2.rectangle()` 在上述匹配位置绘制矩形来标注匹配位置。

使用函数 `numpy.where()` 在函数 `cv2.matchTemplate()` 的输出值中查找指定值，得到的形式为“(行号，列号)”的位置索引。但是，函数 `cv2.rectangle()` 中用于指定顶点的参数所使用的是形式为“(列号，行号)”的位置索引。所以，在使用函数 `cv2.rectangle()` 绘制矩形前，要先将函数 `numpy.where()` 得到的位置索引做“行列互换”。可以使用如下语句实现 `loc` 内行列位置的互换：

```
loc[::-1]
```

如下语句将 `loc` 内的两个元素交换位置：

```
import numpy as np
loc = ([1,2,3,4],[11,12,13,14])
print(loc)
print(loc[::-1])
```

其中，语句 `print(loc)` 所对应的输出为：

```
([1, 2, 3, 4], [11, 12, 13, 14])
```

语句 `print(loc[::-1])` 所对应的输出为：

```
([11, 12, 13, 14], [1, 2, 3, 4])
```

#### 4. 标记匹配图像的位置

函数 `cv2.rectangle()` 可以标记匹配图像的具体位置, 分别指定要标记的原始图像、对角顶点、颜色、矩形边线宽度即可。

关于矩形的对角顶点:

- 其中的一个对角顶点 A 可以通过 for 循环语句从确定的满足条件的“匹配位置集合”内获取。
- 另外一个对角顶点, 可以通过顶点 A 的位置与模板的宽 (w) 和高 (h) 进行运算得到。

因此, 标记各个匹配位置的语句为:

```
for i in 匹配位置集合:
    cv2.rectangle(输入图像,i, (i[0] + w, i[1] + h), 255, 2)
```

**【例 15.3】** 使用模板匹配方式, 标记在输入图像内与模板图像匹配的多个子图像。

根据题目的要求, 编写代码如下:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
img = cv2.imread('lena4.bmp',0)
template = cv2.imread('lena4Temp.bmp',0)
w, h = template.shape[::-1]
res = cv2.matchTemplate(img,template,cv2.TM_CCOEFF_NORMED)
threshold = 0.9
loc = np.where( res >= threshold)
for pt in zip(*loc[::-1]):
    cv2.rectangle(img, pt, (pt[0] + w, pt[1] + h), 255, 1)
plt.imshow(img,cmap = 'gray')
plt.xticks([]), plt.yticks([])
```

本例所使用的输入图像及模板图像如图 15-6 所示, 输入图像由 4 幅 lena 图像构成。



图 15-6 【例 15.3】的模板图像 (左图) 与输入图像 (右图)

运行上述代码, 得到如图 15-7 所示结果, 可以看到输入图像内多个与模板图像匹配的子

图被标记出来。



图 15-7 匹配结果

大家可能已经注意到了，本来在函数 `cv2.rectangle()` 中设置的边界宽度为 1，但实际上标记出来的宽度远远大于 1。这是因为在当前的区域内，存在多个大于当前指定阈值 (0.9) 的情况，所以将它们都做了标记。这样，多个宽度为 1 的矩形就合在了一起，显得边界比较粗。读者可以尝试修改阈值，调整宽度，观察不同的演示效果。